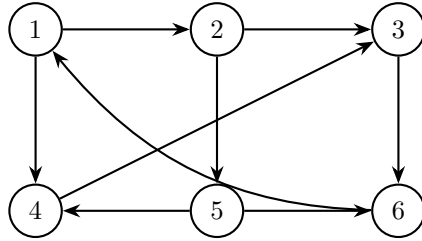


Problem Set 4

Hoa T. Vu

PageRank

Problem 1: PageRank on a Six-Node Graph. Consider the directed graph below with nodes 1, 2, 3, 4, 5, 6 and edges as follows.



- (a) Write the 6×6 transition matrix M , where $M_{ij} = 1/d_j$ if there is an edge $j \rightarrow i$, and 0 otherwise. (d_j is the out-degree of node j .)
- (b) Using teleportation parameter $\epsilon = 0.2$ and $n = 6$, write the modified transition matrix

$$\hat{M} = (1 - \epsilon) M + \frac{\epsilon}{n} J,$$

where J is the 6×6 all-ones matrix.

- (c) Starting from the uniform distribution $\pi^{(0)} = [\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}]^\top$, compute $\pi^{(1)} = \hat{M} \pi^{(0)}$ by hand. Express each entry as a fraction.

Problem 2: Implementing PageRank. Using the same graph from Problem 1, implement PageRank in Python.

- (a) Represent the graph as an adjacency list or edge list. Build the transition matrix M programmatically.
- (b) Implement power iteration with teleportation parameter $\epsilon = 0.2$. Run until $\|\pi^{(t)} - \pi^{(t-1)}\|_1 < 10^{-8}$. Report the final PageRank vector.

Problem 3: PageRank on the Stanford Web Graph. Download the Stanford web graph from the SNAP dataset:

<https://snap.stanford.edu/data/web-Stanford.html>

The dataset `web-Stanford.txt.gz` contains directed edges between pages within the `stanford.edu` domain (281 903 nodes, 2 312 497 edges). Each line (after the header) is a pair `FromNodeId ToNodeId`.

Implement the iterative PageRank algorithm with damping factor $\alpha = 0.85$:

$$\pi^{(t)} = \alpha M \pi^{(t-1)} + \frac{1 - \alpha}{n} \mathbf{1},$$

where M is the column-stochastic transition matrix (handle dangling nodes by redistributing their probability uniformly).

- (a) Parse the edge list and build a sparse representation of the graph. Report the number of nodes and edges after loading.
- (b) Identify all dangling nodes (nodes with no outgoing edges). How many are there?
- (c) Run power iteration until $\|\pi^{(t)} - \pi^{(t-1)}\|_1 < 10^{-6}$. Report the number of iterations required.
- (d) Report the top 100 nodes by PageRank: their node IDs and PageRank values, sorted in descending order.

Implementation notes.

- Build two dictionaries from the edge list: `out[u]` = list of nodes that u links to, and `in[v]` = list of nodes that link to v . The out-degree of u is `len(out[u])`, and the update rule for node v becomes:

```
new_pi[v] = alpha * sum(pi[u] / len(out[u]) for u in in_[v]) \
            + (1 - alpha) / n
```

- As discussedL: to fix dangling nodes, at each iteration, add $\alpha \cdot (\text{sum of dangling-node ranks})/n$ to every entry. A dangling node is any node absent from `out` (or with `len(out[u]) == 0`).